THE COMPLETE JAVASCRIPT
STUDY GUIDE

Beginner to Advanced

Setup • Theory • Examples • Multiple-Choice Questions • Practical Exercises • Answer Key

A hands-on guide for Windows, macOS, and Linux

Table of Contents

This guide is organised into five parts, moving from environment setup through beginner, intermediate, and advanced JavaScript, finishing with capstone projects and a full answer key.

> **How to use this guide:** Read each part in order. Try every example yourself before reading the explanation of what it does. Attempt the multiple-choice questions without looking at the answer key first, then attempt the practical exercises in your own editor. Answers and explanations for every MCQ are collected at the end of the book so you can mark your own work.

Introduction: What Is JavaScript?

JavaScript (often shortened to “JS”) is a lightweight, interpreted (or just-in-time compiled) programming language most famous for making web pages interactive. Alongside HTML (structure) and CSS (style), JavaScript is the third pillar of the web — it is the layer that handles behaviour: responding to clicks, validating forms, updating content without reloading the page, fetching data from servers, animating elements, and much more.

JavaScript is no longer confined to the browser. Thanks to runtimes like Node.js and Deno, JavaScript can also run on servers, build command-line tools, power mobile apps (React Native), control desktop applications (Electron), and even run on microcontrollers. Learning JavaScript well opens the door to front-end development, back-end development, and full-stack engineering.

A Very Short History

JavaScript was created by Brendan Eich in 1995 in just ten days while he was working at Netscape. Despite the name, it has no real relationship to Java — the name was chosen for marketing reasons at a time when Java was extremely popular. In 1997, JavaScript was standardised under the name ECMAScript (ES) by ECMA International, and every official version of JavaScript since then is a specific edition of the ECMAScript standard.

Version	Notable additions
ES5 (2009)	Strict mode, JSON support, Array methods such as forEach, map, filter
ES6 / ES2015	let & const, arrow functions, classes, template literals, promises, modules, destructuring
ES2016–ES2017	Array.includes, exponent operator (**), async/await
ES2018–ES2019	Rest/spread for objects, Array.flat/flatMap, Object.fromEntries
ES2020–ES2022	Optional chaining (?.), nullish coalescing (??), BigInt, top-level await, class fields
ES2023+	Array copying methods (toSorted, toReversed), continued yearly refinements

You do not need to memorise version numbers. What matters is that modern JavaScript (“ES6+”) is what you will write day to day, and this guide teaches the modern syntax throughout while pointing out older patterns you will still encounter in existing code.

How JavaScript Actually Runs

A JavaScript engine is the program that reads your JavaScript code and executes it. Browsers each ship with their own engine: Google Chrome and Node.js use V8, Firefox uses SpiderMonkey, and Safari uses JavaScriptCore. When you write a <script> tag in an HTML page, the browser's engine parses and runs that code. When you run “node app.js” in a terminal, Node.js (which is built on the V8 engine) runs the same language outside of any browser.

JavaScript is single-threaded, meaning it executes one instruction at a time on a single call stack. To stay responsive while waiting on slow operations (like network requests or timers), JavaScript uses an event loop, callback queue, and microtask queue to schedule work asynchronously. We will return to this in detail in Part 3.

Part 0: Getting Your Environment Ready

Before writing any JavaScript, you need three things: a way to run JavaScript on your machine (Node.js), a code editor (Visual Studio Code), and a small set of extensions and command-line tools that make development easier. This section walks through installing all of it on Windows, macOS, and Linux.

0.1 Installing JavaScript (Node.js) on Windows

Node.js bundles the V8 JavaScript engine together with npm (Node Package Manager), so installing Node.js gives you everything needed to run JavaScript from the command line and manage project dependencies.

1. Open your browser and go to the official Node.js website: nodejs.org.
2. Download the "LTS" (Long Term Support) installer for Windows — this is the most stable version and is recommended for almost everyone.
3. Run the downloaded .msi installer. Accept the licence agreement and keep the default installation options, including "Add to PATH", which is checked by default.
4. Click through the installer until it finishes, then restart any open terminal windows (or restart your computer if unsure).
5. Open Command Prompt or PowerShell and confirm the install by running: node -v and npm -v. Both commands should print version numbers.

Expected terminal output after installation (version numbers will vary)

```
node -v
v22.11.0

npm -v
10.9.0
```

> **Tip:** If "node" is not recognised after installation, close and reopen your terminal, or restart your computer, so the updated PATH variable takes effect.

0.2 Installing JavaScript (Node.js) on macOS

There are two common ways to install Node.js on a Mac: the official installer, or the Homebrew package manager. Homebrew is recommended because it also makes future updates easier.

Option A: Using Homebrew (recommended)

1. Open the Terminal app (Applications > Utilities > Terminal, or search with Spotlight).
2. If you do not already have Homebrew installed, install it by pasting the command from brew.sh into your terminal and pressing Enter.
3. Once Homebrew is installed, run: brew install node
4. Confirm the installation with: node -v and npm -v

Terminal commands for macOS (Homebrew route)

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install node
node -v
```

```
npm -v
```

Option B: Official installer

1. Go to nodejs.org in your browser.
2. Download the macOS LTS .pkg installer.
3. Open the downloaded file and follow the installation wizard, using the default settings.
4. Open Terminal and confirm with: node -v and npm -v

0.3 Installing JavaScript (Node.js) on Linux

Most Linux distributions can install Node.js directly from their package manager, though the version available there is sometimes older than the current LTS. Using NodeSource's setup script or a version manager like nvm gives you more control.

Debian / Ubuntu (apt)

Ubuntu/Debian installation via NodeSource

```
sudo apt update
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

Fedora / RHEL (dnf)

Fedora/RHEL installation

```
curl -fsSL https://rpm.nodesource.com/setup_lts.x | sudo bash -
sudo dnf install -y nodejs
node -v
```

Arch Linux (pacman)

Arch Linux installation

```
sudo pacman -Syu nodejs npm
node -v
```

Any distribution: using nvm (Node Version Manager)

nvm lets you install and switch between multiple Node.js versions, which is very useful once you work on more than one project. This approach works the same way on macOS too.

Installing Node.js with nvm

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
# restart your terminal, then:
nvm install --lts
nvm use --lts
node -v
```

0.4 Installing Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight, and extremely popular code editor made by Microsoft. It has excellent built-in JavaScript support and a huge extension marketplace.

Windows

1. Go to code.visualstudio.com.
2. Click the download button — it should automatically detect Windows and offer the correct installer.
3. Run the .exe installer. Accept the licence agreement.
4. On the "Select Additional Tasks" screen, tick "Add to PATH", "Register Code as an editor for supported file types", and optionally "Add 'Open with Code' action" for both files and folders — this lets you right-click a folder and open it directly in VS Code.
5. Finish the installation and launch VS Code.

macOS

1. Go to code.visualstudio.com and download the macOS build (choose Apple Silicon or Intel depending on your Mac's chip).
2. Open the downloaded .zip file, which extracts Visual Studio Code.app.
3. Drag Visual Studio Code.app into your Applications folder.
4. Open VS Code from Applications (or Spotlight search), then open the Command Palette (Cmd+Shift+P) and run "Shell Command: Install 'code' command in PATH" so you can type "code ." from any terminal to open the current folder.

Linux

You can install VS Code from a .deb/.rpm package, or via your distribution's package manager/snap.

Linux installation options for VS Code

```
# Debian/Ubuntu (.deb downloaded from the website)
sudo apt install ./code_*.deb

# Fedora/RHEL (.rpm downloaded from the website)
sudo dnf install ./code-*.rpm

# Or, on any distro with snap enabled:
sudo snap install code --classic
```

0.5 Essential VS Code Extensions for JavaScript

Once VS Code is installed, open the Extensions view (the square icon in the left sidebar, or Ctrl+Shift+X / Cmd+Shift+X) and install the following. These work identically on Windows, macOS, and Linux.

Extension	Why you need it
ESLint	Highlights syntax errors and enforces consistent code style as you type.
Prettier – Code formatter	Automatically formats your code on save so it stays clean and consistent.
Live Server	Spins up a local development server with auto-reload for HTML/CSS/JS projects.

Extension	Why you need it
JavaScript (ES6) code snippets	Adds handy shortcuts that expand into common JS patterns.
Quokka.js	Runs your JavaScript live in the editor and shows values inline (great for learning).
Path Intellisense	Autocompletes filenames when you import modules or reference files.
GitLens	Adds rich Git history and blame information directly inside the editor.
Error Lens	Shows errors and warnings inline next to the code that caused them.

> **Tip:** After installing Prettier and ESLint, open VS Code Settings (Ctrl+, / Cmd+,), search for "format on save", and enable it — your code will be automatically tidied every time you save a file.

0.6 Setting Up a Project and Installing Dependencies

A JavaScript project is typically managed with npm (or an alternative package manager such as yarn or pnpm). npm tracks the libraries ("dependencies") your project needs inside a file called package.json, and stores the actual downloaded code inside a node_modules folder.

Creating a new project

1. Create a new folder for your project and open it in VS Code (File > Open Folder, or "code ." in the terminal).
2. Open the built-in terminal in VS Code (Ctrl+` / Cmd+`).
3. Run: npm init -y — this generates a package.json file with sensible defaults.
4. Create your first file, e.g. index.js, and write some JavaScript in it.
5. Run it with: node index.js

Initialising a project

```
npm init -y
# creates package.json

node index.js
# runs your script with Node.js
```

Installing common dependencies

Dependencies are third-party packages you install so you don't have to write everything from scratch. Use "npm install <package>" (or the shorthand "npm i <package>") for packages your code needs to run, and "npm install -D <package>" (or "-–save-dev") for tools you only need while developing, such as testing frameworks or linters.

Command	What it installs
npm install nodemon --save-dev	Automatically restarts your Node.js app whenever you save a file.
npm install express	A minimal web framework for building servers and APIs in Node.js.
npm install --save-dev jest	A popular testing framework for writing and running unit tests.
npm install --save-dev eslint	Command-line linting to catch errors and enforce style rules.

Command	What it installs
npm install axios	A popular library for making HTTP requests from JavaScript.
npm install dotenv	Loads environment variables from a .env file into your project.

Example package.json

```
// package.json after installing a couple of dependencies
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

> **Reminder:** Never commit the node_modules folder to Git. Create a .gitignore file containing “node_modules” — anyone who clones your project can regenerate it by running “npm install”, since all the required package names and versions are already recorded in package.json.

Running JavaScript in the browser

For front-end work you don't need Node.js at all to execute your code — you can link a script to an HTML file and open it in a browser. This is where the Live Server extension helps, giving you auto-refresh as you edit.

Linking a JavaScript file to an HTML page

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, JavaScript!</h1>
  <script src="script.js"></script>
</body>
</html>
```

A simple script.js linked above

```
// script.js
console.log("Hello from script.js!");
document.querySelector("h1").style.color = "teal";
```

Right-click index.html in VS Code and choose “Open with Live Server” to see it running in your browser with auto-reload, or simply open the console with F12 (Windows/Linux) or Cmd+Option+I (macOS) to see the console.log output.

0.7 Debugging JavaScript Inside VS Code

VS Code has a built-in debugger for Node.js that lets you set breakpoints and step through code without adding console.log statements everywhere.

1. Open the file you want to debug (e.g. index.js).
2. Click in the gutter to the left of a line number to set a breakpoint (a red dot appears).
3. Open the Run and Debug view (Ctrl+Shift+D / Cmd+Shift+D) and click “Run and Debug”, choosing “Node.js” if prompted.
4. Execution will pause at your breakpoint. Use the toolbar to step over, step into, or continue, and hover over variables to see their current values.

> **Tip:** For front-end code running in the browser, use the browser's own DevTools (F12) instead — VS Code's built-in debugger is aimed at Node.js code.

0.8 Version Control Basics with Git

Almost every real JavaScript project is tracked with Git, and hosted on a platform like GitHub. It is worth installing Git alongside Node.js and VS Code from the very beginning.

Command	What it does
git init	Starts tracking the current folder as a new Git repository.
git add .	Stages all changed files, ready to be committed.
git commit -m "message"	Saves a snapshot of the staged changes with a description.
git status	Shows which files have changed since the last commit.
git log	Shows the history of commits.
git clone <url>	Downloads a copy of a remote repository.
git push	Uploads local commits to a remote repository (e.g. GitHub).
git pull	Downloads and merges changes from a remote repository.

> **Tip:** VS Code has Git support built in — the Source Control view (Ctrl+Shift+G / Cmd+Shift+G) lets you stage, commit, and push changes without typing any Git commands, once a repository is initialised.

Part 0 Practice: Check Your Setup Knowledge

Q1. What does installing Node.js give you, beyond the ability to run JavaScript from the command line?

a) A code editor
b) npm, for installing and managing packages
c) A web browser
d) A Git client

Q2. Which command confirms that Node.js was installed correctly?

a) node -v
b) npm start
c) code .

d) git status

Q3. What is the purpose of a .gitignore file containing 'node_modules'?

a) It deletes the node_modules folder

b) It prevents the large, regenerable node_modules folder from being committed to version control

c) It installs dependencies automatically

d) It is required for npm to work at all

Q4. Which VS Code extension automatically formats your code every time you save (once enabled)?

a) GitLens

b) Live Server

c) Prettier

d) Path Intellisense

Q5. What is the difference between a regular dependency and a devDependency in package.json?

a) There is no real difference

b) devDependencies are only needed during development (e.g. testing tools), while dependencies are needed for the app to run

c) dependencies are always free, devDependencies cost money

d) devDependencies are installed globally by default

Part 1: JavaScript Fundamentals (Beginner)

This part assumes no prior JavaScript knowledge. Work through each topic in order, typing out every example yourself rather than copy-pasting — typing code builds muscle memory that reading alone does not.

1.1 Syntax, Statements, and Comments

A JavaScript program is a sequence of statements, usually one per line, optionally ended with a semicolon. JavaScript is case-sensitive, so myVariable and myvariable are two different names. Whitespace outside of strings is generally ignored.

Comments and your first statement

```
// This is a single-line comment

/* This is a
   multi-line comment */

console.log("Hello, world!"); // prints text to the console
```

> **Tip:** console.log() is the most important debugging tool you will use. It prints values to the browser's DevTools console or the terminal, letting you inspect what your code is doing.

1.2 Variables: var, let, and const

Variables store values so you can reuse and change them later. Modern JavaScript uses let for variables that will change and const for variables that will not be reassigned. The older keyword var is still valid but has confusing scoping rules and should be avoided in new code.

Declaring variables

```
let age = 25;
age = 26; // fine, let can be reassigned

const name = "Alex";
// name = "Sam"; // ERROR: cannot reassign a const

var oldStyle = "legacy"; // works, but avoid in modern code
```

Keyword	Behaviour
var	Function-scoped, can be redeclared, hoisted with an initial value of undefined.
let	Block-scoped, can be reassigned but not redeclared in the same scope.
const	Block-scoped, cannot be reassigned; the binding is constant (though object/array contents can still change).

const with objects: the reference is fixed, not the contents

```
const person = { name: "Alex" };
person.name = "Sam"; // allowed — we are changing a property, not reassigning "person"
// person = {}; // ERROR
```

1.3 Data Types

JavaScript has seven primitive types (string, number, boolean, null, undefined, symbol, bigint) plus the object type, which includes arrays, functions, and plain objects. JavaScript is dynamically typed: a variable's type is determined by the value it currently holds, and can change.

The main data types and the typeof operator

```
let str = "hello";      // string
let num = 42;           // number
let isDone = true;      // boolean
let nothing = null;     // null (intentional absence of value)
let notSet;             // undefined (declared, not assigned)
let big = 123n;         // bigint
let sym = Symbol("id"); // symbol
let obj = { a: 1 };     // object
let arr = [1, 2, 3];    // also an object (array)

console.log(typeof str, typeof num, typeof isDone, typeof obj);
```

> **Common trap:** typeof null returns "object" — this is a long-standing bug in JavaScript that can never be fixed without breaking old code, so just memorise it as an exception.

1.4 Operators

Operators let you combine and compare values.

Category	Examples
Arithmetic	+ - * / % (modulus/remainder) ** (exponent)
Assignment	= += -= *= /= %= **=
Comparison	== === != !== > < >= <=
Logical	&& (AND) || (OR) ! (NOT)
Ternary	condition ? valueIfTrue : valueIfFalse
Nullish coalescing	?? — returns the right side only if the left is null or undefined
Optional chaining	?. — safely accesses a property that might not exist

Comparison, nullish coalescing, and optional chaining

```
console.log(5 === "5");   // false: strict equality checks type AND value
console.log(5 == "5");    // true: loose equality converts types first
console.log(10 % 3);      // 1 (remainder)
console.log(2 ** 4);      // 16 (2 to the power of 4)

const user = null;
console.log(user?.name);       // undefined, no error thrown
console.log(user ?? "Guest");  // "Guest" because user is null
```

> **Best practice:** Always prefer === and !== over == and != to avoid unexpected type coercion bugs.

1.5 Strings and Template Literals

Strings represent text and can be written with single quotes, double quotes, or backticks. Backtick strings (template literals) support multi-line text and ${} interpolation, and are the modern preferred style for building strings from variables.

Working with strings

```
const first = "Ada";
const last = "Lovelace";

// old style concatenation
const full1 = first + " " + last;

// modern template literal
const full2 = `${first} ${last}`;
console.log(full2.toUpperCase());   // "ADA LOVELACE"
console.log(full2.length);          // number of characters
console.log(full2.includes("Love"));// true
console.log(full2.slice(0, 3));     // "Ada"
```

1.6 Numbers and the Math Object

Numbers and the built-in Math object

```
console.log(0.1 + 0.2);        // 0.30000000000000004 (floating point precision)
console.log(Number.isInteger(4)); // true
console.log(Math.round(4.7));  // 5
console.log(Math.floor(4.7));  // 4
console.log(Math.ceil(4.1));   // 5
console.log(Math.max(3, 7, 2));// 7
console.log(Math.random());    // random number between 0 and 1
```

1.7 Booleans, Truthy, and Falsy

Every value in JavaScript is either “truthy” or “falsy” when used in a condition. There are exactly six falsy values: false, 0, "" (empty string), null, undefined, and NaN. Everything else, including [] and {}, is truthy.

Truthy and falsy in practice

```
if ("") console.log("never runs");
if ("hello") console.log("runs, non-empty strings are truthy");
if ([]) console.log("runs, empty arrays are truthy!");
```

1.8 Arrays

An array is an ordered list of values, indexed from 0.

Basic array operations

```
const fruits = ["apple", "banana", "cherry"];
console.log(fruits[0]);        // "apple"
console.log(fruits.length);    // 3

fruits.push("date");           // add to the end
fruits.unshift("apricot");     // add to the beginning
fruits.pop();                  // remove from the end
console.log(fruits);
```

```
console.log(fruits.indexOf("banana")); // 1
console.log(fruits.includes("cherry")); // true
```

1.9 Objects

An object stores data as key–value pairs, useful for representing real-world entities.

Creating and using objects

```
const car = {
  make: "Toyota",
  model: "Corolla",
  year: 2022,
  drive() {
    console.log(`${this.make} is driving!`);
  },
};

console.log(car.make);       // dot notation
console.log(car["model"]);   // bracket notation
car.year = 2023;             // update a property
car.color = "blue";          // add a new property
car.drive();                 // "Toyota is driving!"
```

1.10 Control Flow: if/else and switch

if/else and switch statements

```
const score = 82;

if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else {
  console.log("C or below");
}

switch (true) {
  case score >= 90:
    console.log("Grade A");
    break;
  case score >= 80:
    console.log("Grade B");
    break;
  default:
    console.log("Keep practising");
}
```

1.11 Loops

The four most common loop styles

```
for (let i = 0; i < 3; i++) {
  console.log("for loop:", i);
```

```
}

let count = 0;
while (count < 3) {
  console.log("while loop:", count);
  count++;
}

const colors = ["red", "green", "blue"];
for (const color of colors) {
  console.log("for...of:", color);
}

const person = { name: "Kim", age: 30 };
for (const key in person) {
  console.log("for...in:", key, person[key]);
}
```

> **When to use which:** Use for...of to loop over the values of an array or other iterable. Use for...in to loop over the keys of a plain object. Use while when you don't know in advance how many iterations you need.

1.12 Functions Basics

Functions are reusable blocks of code. JavaScript supports several ways to define them.

Function declarations, arrow functions, and function expressions

```
function greet(name) {
  return `Hello, ${name}!`;
}

const greetArrow = (name) => `Hello, ${name}!`;

const greetExpr = function (name) {
  return `Hello, ${name}!`;
};

console.log(greet("Sam"));
console.log(greetArrow("Sam"));
console.log(greetExpr("Sam"));
```

Default parameters

```
function multiply(a, b = 1) { // b defaults to 1 if not provided
  return a * b;
}
console.log(multiply(5));     // 5
console.log(multiply(5, 3));  // 15
```

1.13 Scope

Scope determines where a variable can be accessed. let and const are block-scoped (limited to the nearest { }), while var is function-scoped.

Block scope vs. function scope

```
function demo() {
  if (true) {
    let blockScoped = "only visible inside this block";
    var functionScoped = "visible anywhere in demo()";
  }
  // console.log(blockScoped); // ERROR: not defined here
  console.log(functionScoped); // works fine
}
demo();
```

1.14 Hoisting

Hoisting is JavaScript's behaviour of processing variable and function declarations before running code line by line. Function declarations are hoisted completely (you can call them before they appear in the file). var declarations are hoisted but initialised as undefined. let and const are hoisted too, but remain in a 'temporal dead zone' until their line runs, so accessing them earlier throws an error instead of silently returning undefined.

Hoisting with functions, var, and let

```
console.log(sayHi()); // works! function declarations are fully hoisted
function sayHi() { return "Hi!"; }

console.log(x); // undefined, not an error
var x = 5;

console.log(y); // ERROR: Cannot access 'y' before initialization
let y = 5;
```

1.15 Strict Mode

Adding "use strict"; at the top of a file or function opts into a stricter variant of JavaScript that catches common mistakes, such as accidentally creating a global variable by forgetting to declare it. Code written inside ES6 modules and classes is automatically strict, so you will often benefit from this without writing the directive yourself.

Strict mode catches an accidental global variable

```
"use strict";

function demo() {
  undeclared = 10; // ERROR in strict mode: undeclared is not defined
}
demo();
```

1.16 Comparing Objects and Arrays

Objects and arrays are compared by reference, not by value. Two separately created objects with identical contents are NOT equal, because === checks whether they are the exact same object in memory.

Reference equality vs. content equality

```
console.log([1, 2] === [1, 2]);   // false: two different arrays in memory
```

```
const a = [1, 2];
const b = a;
console.log(a === b); // true: "b" points to the same array as "a"

// To compare contents, compare a serialised form or write a manual check:
console.log(JSON.stringify([1, 2]) === JSON.stringify([1, 2])); // true
```

Part 1 Practice: Multiple-Choice Questions

Q6. Which keyword should you use by default for a variable that will never be reassigned?

a) var
b) let
c) const
d) static

Q7. What does typeof null return?

a) "null"
b) "undefined"
c) "object"
d) "boolean"

Q8. Which of these values is truthy?

a) 0
b) ""
c) []
d) null

Q9. What is the result of 5 === "5"?

a) true
b) false
c) 5
d) an error

Q10. Which loop is best suited to iterating over the values of an array?

a) for...in
b) for...of
c) do...while only
d) switch

Q11. What does the following return: Boolean(undefined ?? "default")?

a) true
b) false
c) undefined
d) "default"

Q12. Which array method adds an element to the end of an array?

a) shift()
b) pop()

c) push()
d) unshift()

Q13. In an object literal, how do you access a property whose name is stored in a variable called key?

a) obj.key
b) obj[key]
c) obj->key
d) obj::key

Q14. What is a function declaration's hoisting behaviour compared to a let variable's?

a) Both are fully usable before their line runs
b) A function declaration is fully usable before its line runs; a let variable throws if accessed too early
c) Neither can be hoisted
d) let is hoisted but a function declaration is not

Q15. Why does [1, 2] === [1, 2] evaluate to false?

a) JavaScript cannot compare arrays at all
b) The two arrays contain different numbers
c) === compares object references, and these are two distinct arrays in memory
d) Arrays must be converted to strings before comparison

Q16. What does "use strict" help catch?

a) Network errors
b) CSS styling mistakes
c) Accidentally creating global variables and other silent mistakes
d) Slow-running loops

Part 1 Practice: Practical Exercises

1. **Temperature converter**

   Write a function celsiusToFahrenheit(c) that converts Celsius to Fahrenheit using the formula (c * 9/5) + 32, and log the result for 0, 20, and 100 degrees.

2. **Even or odd**

   Write a for loop that logs the numbers 1 to 20, printing “even” or “odd” next to each one using the modulus operator.

3. **Shopping list object**

   Create an object representing a shopping list item with name, price, and quantity properties, then log a sentence describing the total cost (price * quantity) using a template literal.

4. **Array cleanup**

   Given const nums = [5, 12, 8, 130, 44]; use push, pop, and includes to add 100, remove the last element, and check whether 12 is present.

5. **FizzBuzz**

   Loop from 1 to 30. Print “Fizz” for multiples of 3, “Buzz” for multiples of 5, “FizzBuzz” for multiples of both, and the number otherwise.

6. **Hoisting prediction**

Write a short script that mixes a var declaration, a let declaration, and a function call before its declaration. Predict the console output on paper first, then run it to check yourself.

7. **Strict-mode bug hunt**

   Write a function that accidentally assigns to an undeclared variable. Run it normally, then add "use strict"; and observe how the error message changes.

Part 2: Intermediate JavaScript

With the fundamentals in place, this part covers the tools you'll use constantly in real projects: advanced functions, array methods, closures, the DOM, and events.

2.1 Arrow Functions in Depth

Arrow functions offer a shorter syntax and, importantly, do not have their own “this” binding — they inherit “this” from the surrounding scope. This makes them ideal for callbacks but unsuitable for object methods that rely on “this”.

Arrow function syntax and the “this” gotcha

```
const add = (a, b) => a + b;              // implicit return, no braces needed
const square = (n) => { return n * n; };  // explicit return with braces
const sayHi = () => console.log("hi");    // no parameters

const obj = {
  value: 10,
  regular: function () { console.log(this.value); },  // 10
  arrow: () => console.log(this.value),                // undefined, "this" is not obj
};
obj.regular();
obj.arrow();
```

2.2 Rest and Spread Operators

Rest parameters and the spread operator

```
function sum(...numbers) {           // rest: gathers arguments into an array
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10

const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // spread: expands an array into individual elements
console.log(arr2); // [1, 2, 3, 4, 5]

const base = { a: 1, b: 2 };
const extended = { ...base, c: 3 }; // spread works on objects too
console.log(extended); // { a: 1, b: 2, c: 3 }
```

2.3 Destructuring

Destructuring objects and arrays

```
const user = { name: "Priya", age: 28, city: "Pune" };
const { name, age } = user;      // object destructuring
console.log(name, age);

const { name: fullName } = user; // renaming while destructuring
console.log(fullName);
```

```
const coordinates = [10, 20, 30];
const [x, y, z] = coordinates;   // array destructuring
console.log(x, y, z);

function printUser({ name, age }) { // destructuring straight in a parameter list
  console.log(`${name} is ${age}`);
}
printUser(user);
```

2.4 Higher-Order Functions and Array Methods

A higher-order function either takes another function as an argument or returns one. JavaScript's array methods lean heavily on this pattern and are essential for writing clean, declarative code instead of manual loops.

Method	Purpose
map()	Transforms every element and returns a new array of the same length.
filter()	Returns a new array containing only elements that pass a test.
reduce()	Combines all elements into a single value (a sum, an object, etc.).
forEach()	Runs a function for each element; does not return a new array.
find()	Returns the first element that matches a condition, or undefined.
some() / every()	Return true/false depending on whether any/all elements pass a test.
sort()	Sorts the array in place (be careful — default sort is lexicographic).

The core array methods in action

```
const nums = [1, 2, 3, 4, 5, 6];

const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10, 12]

const evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6]

const total = nums.reduce((acc, n) => acc + n, 0);
console.log(total); // 21

const firstBig = nums.find(n => n > 4);
console.log(firstBig); // 5

console.log(nums.every(n => n > 0)); // true
console.log(nums.some(n => n > 5));  // true
```

> **Common trap:** [10, 2, 1].sort() gives [1, 10, 2] by default because sort() converts elements to strings first. Pass a compare function — (a, b) => a - b — to sort numbers correctly.

2.5 Closures

A closure is a function that “remembers” the variables from the scope in which it was created, even after that outer function has finished running. Closures are the mechanism behind private variables and many popular patterns.

A classic closure: a private counter

```
function makeCounter() {
  let count = 0;              // "count" is captured by the closure
  return function () {
    count++;
    return count;
  };
}

const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3, "count" persisted between calls
```

2.6 The “this” Keyword

"this" refers to the object that is currently executing the function. Its value depends on HOW a function is called, not where it is defined (except for arrow functions, which inherit "this" from their surrounding scope).

How “this” changes depending on how a function is invoked

```
const dog = {
  name: "Rex",
  bark() {
    console.log(`${this.name} says woof!`); // "this" = dog, because dog.bark() called
it
  },
};
dog.bark(); // "Rex says woof!"

const bark = dog.bark;
// bark(); // "this" is now undefined/global — loses the connection to dog

const boundBark = dog.bark.bind(dog); // bind() locks "this" permanently
boundBark(); // "Rex says woof!"
```

2.7 Error Handling

try/catch/finally and throwing custom errors

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}

try {
  console.log(divide(10, 0));
} catch (error) {
```

```
  console.log("Something went wrong:", error.message);
} finally {
  console.log("This always runs, error or not.");
}
```

Creating a custom error type

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

function setAge(age) {
  if (age < 0) throw new ValidationError("Age cannot be negative");
  return age;
}

try {
  setAge(-5);
} catch (err) {
  if (err instanceof ValidationError) {
    console.log("Validation failed:", err.message);
  }
}
```

2.8 JSON

JSON (JavaScript Object Notation) is a lightweight text format for exchanging data, commonly used when talking to servers/APIs. JavaScript provides built-in methods to convert between objects and JSON text.

Converting objects to JSON and back

```
const user = { name: "Lee", age: 34, hobbies: ["chess", "hiking"] };

const jsonString = JSON.stringify(user);
console.log(jsonString); // '{"name":"Lee","age":34,"hobbies":["chess","hiking"]}'

const parsedBack = JSON.parse(jsonString);
console.log(parsedBack.name); // "Lee"
```

2.9 Dates

Working with the built-in Date object

```
const now = new Date();
console.log(now.getFullYear());
console.log(now.getMonth());     // 0-indexed! January = 0
console.log(now.getDate());

const birthday = new Date("2000-05-15");
console.log(birthday.toDateString());
```

2.10 Regular Expressions

Basic pattern matching with regular expressions

```
const pattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
console.log(pattern.test("hello@example.com")); // true
console.log(pattern.test("not-an-email"));      // false

const text = "The rain in Spain";
console.log(text.match(/ain/g));   // ["ain", "ain"]
console.log(text.replace(/rain/, "sun")); // "The sun in Spain"
```

2.11 The DOM (Document Object Model)

The DOM is the browser's in-memory, tree-shaped representation of an HTML page. JavaScript can read and change the DOM to update what the user sees, without reloading the page.

HTML used by the example below

```
<button id="myBtn">Click me</button>
<p id="output"></p>
```

Selecting, creating, and styling elements

```
const button = document.getElementById("myBtn");
const output = document.querySelector("#output");

const heading = document.createElement("h2");
heading.textContent = "Created with JavaScript!";
document.body.appendChild(heading);

output.style.color = "purple";
output.classList.add("highlight");
```

Method	Use
document.getElementById(id)	Selects a single element by its id attribute.
document.querySelector(selector)	Selects the first element matching any CSS selector.
document.querySelectorAll(selector)	Selects all matching elements as a NodeList.
element.textContent / innerHTML	Reads or sets the text/HTML content of an element.
element.classList.add/remove/toggle	Manages an element's CSS classes.
element.setAttribute(name, value)	Sets an HTML attribute on an element.

2.12 Events and Event Listeners

Listening for and removing events

```
button.addEventListener("click", function (event) {
  output.textContent = "Button was clicked!";
  console.log(event.type); // "click"
});

// Removing a listener requires a named function reference
```

```
function handleClick() { console.log("clicked"); }
button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick);
```

Event Bubbling, Capturing, and Delegation

When an event fires on an element, it “bubbles” upward through its ancestors by default (capturing is the reverse, top-down phase, rarely used). Event delegation exploits bubbling: instead of attaching a listener to every child element, you attach one listener to a parent and check which child triggered it — much more efficient for lists that change dynamically.

Event delegation on a list

```
document.getElementById("list").addEventListener("click", function (event) {
  if (event.target.tagName === "LI") {
    console.log("You clicked:", event.target.textContent);
  }
});
// One listener on the parent handles clicks on any current or future <li>
```

2.13 Useful Object and Array Utility Methods

Beyond the core array methods, a handful of static utility methods come up constantly when working with objects and collections.

Object.keys, values, entries, freeze, and assign

```
const user = { name: "Zoe", age: 29 };

console.log(Object.keys(user));    // ["name", "age"]
console.log(Object.values(user));  // ["Zoe", 29]
console.log(Object.entries(user)); // [["name","Zoe"], ["age",29]]

const frozen = Object.freeze({ a: 1 });
// frozen.a = 2; // silently ignored (or throws in strict mode)
console.log(frozen.a); // still 1

const merged = Object.assign({}, { a: 1 }, { b: 2 });
console.log(merged); // { a: 1, b: 2 }
```

flat, flatMap, and Array.from

```
const nested = [1, [2, 3], [4, [5, 6]]];
console.log(nested.flat());       // [1, 2, 3, 4, [5, 6]]
console.log(nested.flat(2));      // [1, 2, 3, 4, 5, 6]

const words = ["hello world", "foo bar"];
console.log(words.flatMap(w => w.split(" ")));
// ["hello", "world", "foo", "bar"]

console.log(Array.from({ length: 3 }, (_, i) => i * 2)); // [0, 2, 4]
```

2.14 Copying Data Safely: Shallow vs. Deep Copies

Spreading an array or object only creates a shallow copy — nested objects/arrays inside it are still shared references. For a true deep copy, use structuredClone() (built into modern browsers and Node.js) or JSON.parse(JSON.stringify(...)) as an older fallback (which loses functions and special types).

Shallow copies still share nested references

```
const original = { name: "Kai", address: { city: "Lagos" } };
const shallow = { ...original };
shallow.address.city = "Abuja";
console.log(original.address.city); // "Abuja" — the nested object was shared!

const deep = structuredClone(original);
deep.address.city = "Nairobi";
console.log(original.address.city); // unaffected, still whatever it was before
```

2.15 Web Storage: localStorage and sessionStorage

Browsers provide two simple key-value stores for persisting small amounts of data on the user's device. localStorage persists until explicitly cleared; sessionStorage is cleared when the tab closes. Both only store strings, so objects must be serialised with JSON.stringify/parse.

Storing and retrieving data with localStorage

```
localStorage.setItem("username", "sam");
console.log(localStorage.getItem("username")); // "sam"

const cart = { items: ["book", "pen"] };
localStorage.setItem("cart", JSON.stringify(cart));
const restoredCart = JSON.parse(localStorage.getItem("cart"));
console.log(restoredCart.items); // ["book", "pen"]

localStorage.removeItem("username");
```

Part 2 Practice: Multiple-Choice Questions

Q17. Which array method returns a brand-new array without modifying the original?

a) sort()
b) map()
c) push()
d) splice()

Q18. What value does reduce() return in nums.reduce((acc, n) => acc + n, 0) for nums = [1,2,3]?

a) [1,2,3]
b) 6
c) 0
d) undefined

Q19. Why do arrow functions behave differently from regular functions regarding “this”?

a) Arrow functions always set this to the global object
b) Arrow functions do not have their own this and inherit it from the enclosing scope
c) Arrow functions require bind() to work at all

d) There is no difference

Q20. What does a closure allow a function to do?

a) Run faster than normal functions
b) Access variables from its outer scope even after that outer function has returned
c) Automatically catch errors
d) Avoid using parameters

Q21. Which method converts a JavaScript object into a JSON string?

a) JSON.parse()
b) JSON.stringify()
c) Object.toJSON()
d) String(object)

Q22. In the DOM, which method selects ALL elements matching a CSS selector?

a) document.getElementById()
b) document.querySelector()
c) document.querySelectorAll()
d) document.getAll()

Q23. What is event delegation primarily used for?

a) Preventing all events from firing
b) Attaching one listener to a parent to handle events from many current or future children
c) Making events run asynchronously
d) Converting click events into keyboard events

Q24. Why does modifying shallow.address.city also change original.address.city after const shallow = { ...original }?

a) Spread always deep-copies objects
b) The spread only copies top-level properties; nested objects are still shared by reference
c) This is a bug in JavaScript that will be fixed
d) address is a special read-only property

Q25. What is the key difference between localStorage and sessionStorage?

a) sessionStorage can store objects directly, localStorage cannot
b) localStorage persists until cleared; sessionStorage clears when the tab closes
c) They are identical in every way
d) localStorage only works in Node.js

Q26. What does Object.freeze() do to an object?

a) Deletes all its properties
b) Converts it to a JSON string
c) Prevents its properties from being changed, added, or removed
d) Makes it iterable

Part 2 Practice: Practical Exercises

1. **Array pipeline**

Given an array of product objects with name and price, use filter to keep items under $50, map to extract just the names, and log the result.

2. **Counter closure**

   Write a makeCounter() function (like the example above) but extend it to also return a reset() method that sets the count back to zero.

3. **Form validator**

   Build a small HTML form with a text input and a submit button. Use JavaScript to prevent submission and show an error message if the input is empty.

4. **To-do list with delegation**

   Render a <ul> of to-do items. Add one click listener on the <ul> that toggles a 'completed' class on whichever <li> was clicked.

5. **Safe JSON parsing**

   Write a function safeParse(jsonString) that uses try/catch to return the parsed object, or null if the string is invalid JSON.

6. **Deep vs. shallow copy demo**

   Create an object with a nested array. Make a shallow copy with spread and a deep copy with structuredClone. Mutate the nested array in each copy and log the original to show the difference.

7. **Persisted notes app**

   Build a small textarea that saves its content to localStorage on every keystroke and restores it automatically when the page reloads.

Part 3: Advanced JavaScript

This part covers the concepts that separate comfortable JavaScript users from confident ones: prototypes, classes, asynchronous programming, modules, and the event loop.

3.1 Prototypes and Prototypal Inheritance

Every JavaScript object has a hidden internal link to another object called its prototype, which it can inherit properties and methods from. This is different from classical (class-based) inheritance in languages like Java, and is the mechanism that ES6 classes are built on top of.

Prototype-based inheritance with Object.create

```
const animal = {
  eats: true,
  walk() { console.log("Animal walks"); },
};

const rabbit = Object.create(animal); // rabbit's prototype is "animal"
rabbit.jumps = true;

console.log(rabbit.eats);  // true, inherited from animal
rabbit.walk();              // "Animal walks", inherited method
console.log(Object.getPrototypeOf(rabbit) === animal); // true
```

3.2 ES6 Classes

Classes are “syntactic sugar” over JavaScript's prototype system — they don't introduce a new inheritance model, but they make it much easier to read and write.

Class inheritance with extends and super

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    console.log(`${this.name} makes a sound.`);
  }
}

class Dog extends Animal {
  speak() {
    super.speak();               // call the parent method too
    console.log(`${this.name} barks.`);
  }
}

const d = new Dog("Rex");
d.speak();
// Rex makes a sound.
// Rex barks.
```

Private fields, getters, and static methods

```
class BankAccount {
  #balance = 0; // private field (only accessible inside the class)

  constructor(owner) {
    this.owner = owner;
  }

  deposit(amount) {
    this.#balance += amount;
    return this.#balance;
  }

  get balance() {       // getter
    return this.#balance;
  }

  static bankName() {   // static method belongs to the class, not an instance
    return "JS Bank";
  }
}

const acc = new BankAccount("Mia");
acc.deposit(100);
console.log(acc.balance);         // 100
console.log(BankAccount.bankName()); // "JS Bank"
// console.log(acc.#balance);      // ERROR: private fields are not accessible outside
the class
```

3.3 Asynchronous JavaScript: Callbacks

Many operations (reading files, network requests, timers) take time. JavaScript historically handled this with callback functions, executed once the operation finishes.

A basic callback with setTimeout

```
console.log("Start");

setTimeout(() => {
  console.log("This runs after 2 seconds");
}, 2000);

console.log("End");
// Output order: Start, End, This runs after 2 seconds
```

> **Callback hell:** Nesting many callbacks inside one another (e.g. one network request depending on the result of another) produces deeply indented, hard-to-read code often called “callback hell”. Promises and async/await were introduced specifically to solve this.

3.4 Promises

A Promise represents a value that is not available yet but will be at some point — either resolved (success) or rejected (failure). Promises can be chained with .then() and .catch(), avoiding deep nesting.

Creating and chaining Promises

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) {
        resolve({ id, name: "User " + id });
      } else {
        reject(new Error("Invalid id"));
      }
    }, 1000);
  });
}

fetchUser(1)
  .then((user) => {
    console.log("Got user:", user);
    return fetchUser(2); // chaining
  })
  .then((user2) => console.log("Got second user:", user2))
  .catch((err) => console.log("Error:", err.message))
  .finally(() => console.log("Done, success or not"));
```

Running promises in parallel

```
Promise.all([fetchUser(1), fetchUser(2), fetchUser(3)])
  .then((users) => console.log("All resolved:", users))
  .catch((err) => console.log("At least one failed:", err.message));
// Promise.all rejects immediately if ANY promise rejects.
// Promise.allSettled() instead waits for every promise and never rejects.
```

3.5 Async/Await

async/await is modern syntax built on top of Promises that lets asynchronous code read like ordinary, synchronous code. A function marked async always returns a Promise, and await pauses execution inside that function until the awaited Promise settles.

Rewriting the promise chain above with async/await

```
async function loadUsers() {
  try {
    const user1 = await fetchUser(1);
    console.log("First:", user1);
    const user2 = await fetchUser(2);
    console.log("Second:", user2);
  } catch (err) {
    console.log("Something failed:", err.message);
  }
}

loadUsers();
```

3.6 The Fetch API

Fetching data from a real API

```
async function getPost(id) {
  const response = await fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);
  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }
  const data = await response.json();
  return data;
}

getPost(1)
  .then((post) => console.log(post.title))
  .catch((err) => console.log("Failed to load post:", err.message));
```

3.7 The Event Loop, Call Stack, and Task Queues

JavaScript executes code on a single call stack. Asynchronous work (timers, network responses, promise callbacks) doesn't run on the stack directly — it waits in queues and is only pushed onto the stack once the stack is empty. Promise callbacks (“microtasks”) are always processed before setTimeout callbacks (“macrotasks”), even if the timer delay is 0.

Demonstrating the event loop's ordering rules

```
console.log("1: sync");

setTimeout(() => console.log("2: macrotask (setTimeout)"), 0);

Promise.resolve().then(() => console.log("3: microtask (promise)"));

console.log("4: sync");

// Output order: 1, 4, 3, 2
// Synchronous code always runs first, then all microtasks, then macrotasks.
```

3.8 Modules

Modules let you split code across multiple files and explicitly share only what's needed. Modern JavaScript uses ES Modules (import/export); Node.js also supports the older CommonJS system (require/module.exports).

Exporting from a module

```
// mathUtils.js (ES Module)
export function add(a, b) { return a + b; }
export const PI = 3.14159;
export default function multiply(a, b) { return a * b; }
```

Importing named and default exports

```
// app.js
import multiply, { add, PI } from "./mathUtils.js";

console.log(add(2, 3));      // 5
```

```
console.log(PI);              // 3.14159
console.log(multiply(2, 3)); // 6
```

CommonJS syntax, still common in Node.js

```
// CommonJS style, used natively by older Node.js code
const { add } = require("./mathUtils.js");
module.exports = { subtract: (a, b) => a - b };
```

> **Note:** To use ES Modules in Node.js, either name files with a ".mjs" extension or add "type": "module" to package.json. In the browser, add type="module" to your <script> tag.

3.9 Iterators, Generators, Map, and Set

A simple generator

```
function* countTo(n) {   // a generator function, marked with *
  for (let i = 1; i <= n; i++) {
    yield i;               // pauses and returns a value each time
  }
}

for (const num of countTo(3)) {
  console.log(num); // 1, 2, 3
}
```

Set (unique values) and Map (key-value pairs with any key type)

```
const uniqueIds = new Set([1, 2, 2, 3, 3, 3]);
console.log(uniqueIds);        // Set(3) {1, 2, 3}
console.log(uniqueIds.has(2)); // true

const userRoles = new Map();
userRoles.set("alice", "admin");
userRoles.set("bob", "editor");
console.log(userRoles.get("alice")); // "admin"
for (const [user, role] of userRoles) {
  console.log(user, "->", role);
}
```

3.10 Introduction to Node.js

Node.js provides built-in modules for working outside the browser, such as reading files and creating web servers.

Reading and writing files with Node's fs module

```
const fs = require('fs');

fs.writeFileSync('notes.txt', 'Hello from Node.js!');
const content = fs.readFileSync('notes.txt', 'utf8');
console.log(content);
```

A minimal HTTP server with Node's built-in http module

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from a Node.js server!');
});

server.listen(3000, () => console.log('Server running on port 3000'));
```

3.11 Design Patterns You'll Recognise

Pattern	Idea
Module pattern	Group related code and expose only what's needed, hiding internal details (achieved naturally with ES Modules or closures).
Singleton	Ensure only one instance of an object/config exists throughout an app.
Observer / Pub-Sub	One or more objects (subscribers) get notified automatically when another object's state changes (this is how addEventListener conceptually works).
Factory	A function that creates and returns objects, hiding the construction details from the caller.

3.12 Testing Basics

Automated tests check that your code behaves correctly and keep working as you make changes. Jest is one of the most widely used JavaScript testing frameworks.

A minimal Jest unit test (run with: npx jest)

```
// sum.js
function sum(a, b) { return a + b; }
module.exports = sum;

// sum.test.js
const sum = require("./sum");

test("adds 1 + 2 to equal 3", () => {
  expect(sum(1, 2)).toBe(3);
});
```

3.13 Best Practices and Clean Code

- Use const by default, let when reassignment is needed, and avoid var.
- Prefer strict equality (=== / !==) over loose equality.
- Give variables and functions descriptive names (isVisible, calculateTotal) rather than single letters.
- Keep functions small and focused on a single task.
- Handle errors explicitly with try/catch around code that can fail, especially async operations.
- Avoid deeply nested callbacks; prefer async/await for asynchronous code.
- Comment on why code does something, not what it does when the code is already clear.
- Run a linter (ESLint) and formatter (Prettier) consistently across a project.

3.14 Currying and Function Composition

Currying transforms a function that takes multiple arguments into a sequence of functions that each take one argument. Composition combines small, single-purpose functions into a larger pipeline. Both patterns lean on closures and higher-order functions.

Currying and composing small functions

```
function curriedAdd(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
}
console.log(curriedAdd(1)(2)(3)); // 6

const compose = (...fns) => (input) => fns.reduceRight((acc, fn) => fn(acc), input);
const double = (n) => n * 2;
const addOne = (n) => n + 1;
const doubleThenAddOne = compose(addOne, double);
console.log(doubleThenAddOne(5)); // double(5)=10, addOne(10)=11
```

3.15 Memory Management and Garbage Collection

JavaScript automatically manages memory using a garbage collector, most commonly implemented with a 'mark-and-sweep' algorithm: the engine periodically checks which objects are still reachable from the global scope or an active call stack, and frees memory for anything that is not. You rarely need to manage memory manually, but a few patterns can cause memory leaks: forgotten timers/intervals, event listeners that are never removed, and references kept alive inside long-lived closures or caches.

A closure that unintentionally keeps a large array alive

```
function startTimer() {
  const bigData = new Array(1000000).fill("x");
  const id = setInterval(() => {
    console.log(bigData.length); // "bigData" is kept alive forever by this closure
  }, 5000);
  // clearInterval(id) is never called — this is a common source of leaks
}
```

3.16 Debugging with the Browser DevTools

- console.log(), console.table(), and console.error() are quick first tools — console.table() is especially useful for arrays of objects.
- Breakpoints: click a line number in the Sources tab (Chrome) to pause execution there and inspect variables.
- The debugger; statement pauses execution at that exact line when DevTools is open, without needing to click in the UI.
- The Network tab shows every request your page makes, including status codes and response bodies — essential when debugging fetch() calls.

- The Elements/Inspector tab lets you see and edit the live DOM and CSS, which is useful for confirming your JavaScript changed what you expected.

Part 3 Practice: Multiple-Choice Questions

Q27. What does every JavaScript object have a hidden link to, forming the basis of inheritance?

a) A constructor table
b) A prototype
c) A schema
d) A class map

Q28. What does the keyword 'super' do inside a subclass method?

a) Deletes the parent class
b) Calls the corresponding method or constructor on the parent class
c) Creates a new instance of the subclass
d) Marks a method as private

Q29. What does an async function always return?

a) undefined
b) The awaited value directly
c) A Promise
d) A callback

Q30. Given the event loop rules, which callback runs first: a Promise.then() or a setTimeout(fn, 0)?

a) setTimeout always wins
b) They run at the exact same time
c) The Promise.then() callback (a microtask) runs first
d) It's random

Q31. Which built-in structure guarantees unique values with no duplicates?

a) Array
b) Object
c) Set
d) Map

Q32. In ES Modules, how do you import the default export from a file?

a) import { default } from './file.js';
b) import anyName from './file.js';
c) require('./file.js').default;
d) import * as default from './file.js';

Q33. What problem do Promises and async/await primarily solve?

a) Making JavaScript multi-threaded
b) Deeply nested, hard-to-read callback chains (“callback hell”)
c) Removing the need for error handling
d) Making variables block-scoped

Q34. What does currying transform a multi-argument function into?

a) A single function that ignores all but the first argument
b) A sequence of functions that each take one argument
c) An asynchronous function
d) A class

Q35. Which of the following is a common cause of memory leaks in JavaScript?

a) Using const instead of let
b) Forgetting to clear an interval/timer or remove an event listener that is no longer needed
c) Using arrow functions
d) Declaring too many variables with let

Q36. Which DevTools feature shows every network request your page makes, including status codes?

a) The Console tab
b) The Elements tab
c) The Network tab
d) The Sources tab only

Part 3 Practice: Practical Exercises

1. **Build a class hierarchy**

   Create a Shape base class with an area() method, then Circle and Rectangle subclasses that override area() appropriately. Instantiate each and log their areas.

2. **Promise-based delay**

   Write a function wait(ms) that returns a Promise resolving after ms milliseconds, then use async/await to log 'A', wait 1 second, and log 'B'.

3. **Fetch and display**

   Using the Fetch API, retrieve data from https://jsonplaceholder.typicode.com/users and log each user's name and email using a for...of loop.

4. **Custom module**

   Create a small module file exporting functions to validate an email and a phone number, then import and use both functions from a separate file.

5. **Simple pub-sub**

   Implement a small EventEmitter-style object with on(event, callback) and emit(event, data) methods, then use it to log a message whenever a custom 'orderPlaced' event is emitted.

6. **Curried multiply**

   Write a curried function multiply(a)(b)(c) that returns the product of all three numbers, then use compose() to chain it with another function of your choice.

7. **Find the leak**

   Given a function that starts a setInterval capturing a large array in its closure, rewrite it so the interval is stored, cleared with clearInterval() after 10 seconds, and confirm (conceptually) why this prevents a leak.

Part 4: Capstone Projects

Reading and small exercises build recognition; finishing a full project builds real skill. Pick at least one project from each tier below and build it from scratch using only what you've learned in this guide, adding features as you go.

4.1 Beginner Capstone Projects

1. **Calculator**

   Build a simple calculator with HTML buttons for digits and operators. Use JavaScript to track the current expression and evaluate it when '=' is pressed.

2. **Number guessing game**

   Generate a random number between 1 and 100 with Math.random(). Let the user guess via a text input, and tell them 'too high', 'too low', or 'correct' using conditional logic.

3. **Digital clock**

   Use setInterval() and the Date object to display the current time, updating every second, formatted as HH:MM:SS.

4.2 Intermediate Capstone Projects

1. **To-do list app**

   Build a to-do list where users can add, mark as complete, and delete tasks. Store the array of tasks in memory and re-render the list using array methods (map, filter) whenever it changes.

2. **Weather dashboard**

   Use the Fetch API to call a public weather API and display the current temperature and conditions for a city the user types in, handling loading and error states.

3. **Quiz app**

   Store an array of question objects (question, options, correctIndex). Display one question at a time, track the score, and show a final results screen using DOM manipulation and event listeners.

4.3 Advanced Capstone Projects

1. **Expense tracker with classes**

   Model Expense and Budget as ES6 classes with private fields for amounts. Persist data with localStorage (JSON.stringify/parse) so it survives a page refresh.

2. **Node.js REST API**

   Using Node.js and Express, build a simple API with GET/POST/PUT/DELETE routes for a resource such as 'books', storing data in memory or a JSON file, with async/await used throughout.

3. **Real-time search with debouncing**

   Build a search box that fetches results from an API as the user types, using a debounce function (built with setTimeout and closures) so a request only fires after the user pauses typing.

4.4 Where to Go Next

- TypeScript — adds static typing on top of JavaScript, catching many bugs before you even run your code.
- A front-end framework such as React, Vue, or Svelte — builds on everything in this guide, especially functions, arrays, and the DOM.
- Node.js frameworks such as Express or Fastify for back-end development.
- Git and GitHub for version control and collaboration — essential for any real project.
- Testing frameworks (Jest, Vitest) and end-to-end tools (Playwright, Cypress) to build confidence in larger codebases.

> **Final thought:** JavaScript rewards practice far more than memorisation. Revisit the practical exercises in this guide after a few weeks — you'll likely solve them faster and write cleaner code the second time around.

Quick Reference: Cheat Sheets

Use these tables as a fast lookup once you already understand the concepts behind them — they are not a substitute for the explanations earlier in this guide.

String Methods Cheat Sheet

Method	What it does
str.length	Number of characters in the string.
str.toUpperCase() / toLowerCase()	Returns a new string in upper/lower case.
str.trim()	Removes whitespace from both ends.
str.slice(start, end)	Extracts a section of the string (end is exclusive).
str.split(separator)	Splits a string into an array using the given separator.
str.includes(sub)	Returns true/false depending on whether sub appears.
str.replace(old, new)	Replaces the first match of old with new.
str.padStart(len, char) / padEnd	Pads a string to a target length.

Array Methods Cheat Sheet

Method	What it does
push() / pop()	Add/remove from the end of the array.
shift() / unshift()	Remove/add from the beginning of the array.
map(fn)	Transforms every element, returns a new array.
filter(fn)	Returns a new array of elements passing a test.
reduce(fn, initial)	Combines all elements into a single value.
find(fn) / findIndex(fn)	Returns the first matching element / its index.
includes(value)	Returns true/false if the value exists in the array.
sort(compareFn)	Sorts in place; always pass a compare function for numbers.
slice(start, end)	Returns a shallow copy of a portion, without mutating.
splice(start, count, ...items)	Mutates the array: removes/inserts elements in place.
flat(depth) / flatMap(fn)	Flattens nested arrays / maps then flattens one level.
join(separator)	Combines all elements into a single string.

Object Methods Cheat Sheet

Method	What it does
Object.keys(obj)	Array of an object's own property names.
Object.values(obj)	Array of an object's own property values.
Object.entries(obj)	Array of [key, value] pairs.
Object.assign(target, ...sources)	Copies properties from sources into target.
Object.freeze(obj)	Prevents further changes to the object.
Object.create(proto)	Creates a new object with the given prototype.
obj.hasOwnProperty(key)	Checks if a property exists directly on the object.

Common HTTP Status Codes (for Fetch/APIs)

Code	Meaning
200 OK	The request succeeded.
201 Created	A new resource was successfully created.
204 No Content	Success, but there is no body to return.
400 Bad Request	The request was malformed or invalid.
401 Unauthorized	Authentication is required or has failed.
403 Forbidden	Authenticated, but not allowed to access this resource.
404 Not Found	The requested resource doesn't exist.
500 Internal Server Error	Something went wrong on the server.

Regular Expression Quick Reference

Pattern	Matches
\d	Any digit (0–9).
\w	Any word character (letters, digits, underscore).
\s	Any whitespace character.
.	Any character except a newline.
^ / $	Start / end of the string.
*****	Zero or more of the previous token.
+	One or more of the previous token.
?	Zero or one of the previous token (makes it optional).
{2,4}	Between 2 and 4 repetitions of the previous token.
[abc]	Any one of the characters a, b, or c.

Pattern	Matches
(?:...)	A non-capturing group, used for grouping without saving a match.

VS Code Keyboard Shortcuts Worth Learning

Shortcut (Win/Linux)	Action
Ctrl+P	Quickly open any file by typing its name.
Ctrl+Shift+P	Open the Command Palette (run any VS Code command).
Ctrl+`	Open/close the integrated terminal.
Ctrl+/	Toggle a comment on the current line(s).
Ctrl+D	Select the next occurrence of the current word (multi-cursor editing).
Ctrl+Shift+K	Delete the current line.
Alt+Up / Alt+Down	Move the current line up or down.
F12	Go to the definition of a function or variable.

> **Note:** On macOS, replace Ctrl with Cmd for most of these shortcuts.

Common npm Commands

Command	What it does
npm init -y	Creates a package.json with default values.
npm install <pkg>	Installs a package and adds it to dependencies.
npm install -D <pkg>	Installs a package as a dev-only dependency.
npm install	Installs everything listed in package.json.
npm uninstall <pkg>	Removes a package.
npm run <script>	Runs a script defined in package.json's "scripts" section.
npm update	Updates installed packages to the latest allowed version.
npx <tool>	Runs a package's CLI tool without installing it globally.

Common JavaScript Interview Questions

These short-answer questions are commonly asked in junior to mid-level JavaScript interviews. Try answering each one aloud in your own words before reading the model answer — explaining concepts out loud is one of the best ways to find gaps in your understanding.

What is the difference between == and ===?

== compares values after converting them to the same type if needed (loose equality); === compares both type and value without any conversion (strict equality). Modern style guides recommend always using ===.

What is a closure, in one sentence?

A closure is a function that retains access to the variables from the scope in which it was created, even after that outer scope has finished executing.

What is the difference between null and undefined?

undefined means a variable has been declared but not yet assigned a value; null is an intentional value representing 'no value', explicitly assigned by the programmer.

What is the event loop?

The mechanism that allows JavaScript, despite being single-threaded, to handle asynchronous operations: it continuously checks whether the call stack is empty and, if so, moves the next callback from the microtask or macrotask queue onto the stack to run.

What is the difference between let, const, and var?

var is function-scoped and hoisted with an initial value of undefined; let and const are block-scoped, and const additionally cannot be reassigned after its initial value is set.

What does 'this' refer to inside a regular function versus an arrow function?

In a regular function, 'this' depends on how the function was called (its 'receiver'). In an arrow function, 'this' is inherited lexically from the surrounding scope at the time the arrow function was defined.

What is the difference between synchronous and asynchronous code?

Synchronous code runs one instruction at a time, each waiting for the previous one to finish. Asynchronous code can start an operation (like a network request) and continue running other code while it waits for that operation to complete.

What is a Promise?

An object representing the eventual completion or failure of an asynchronous operation, with three states: pending, fulfilled, and rejected.

What is the difference between Promise.all and Promise.allSettled?

Promise.all resolves with all results only if every promise succeeds, and rejects immediately if any one fails. Promise.allSettled always waits for every promise and returns the outcome (fulfilled or rejected) of each, never rejecting itself.

What is event bubbling?

When an event fires on an element, it propagates ('bubbles') upward through that element's ancestors in the DOM, triggering any matching listeners along the way, unless propagation is explicitly stopped.

What is the difference between == null checks and optional chaining (?.)?

Optional chaining (?.) short-circuits and returns undefined when accessing a property on null/undefined, without throwing an error, letting you safely read deeply nested properties in one expression instead of writing multiple manual null checks.

What is a higher-order function?

A function that takes one or more functions as arguments, returns a function, or both. map, filter, and reduce are common built-in examples.

What is the difference between call, apply, and bind?

All three let you explicitly set what 'this' refers to inside a function. call() invokes the function immediately with arguments listed individually; apply() invokes it immediately with arguments as an array; bind() returns a new function with 'this' permanently set, without invoking it immediately.

What is a pure function?

A function that, given the same input, always returns the same output and has no side effects (it doesn't modify anything outside its own scope, such as global variables or its arguments).

What is NaN, and how do you check for it?

NaN ('Not a Number') is a special numeric value representing an invalid math operation, such as 0/0 or parsing a non-numeric string. Because NaN !== NaN, use Number.isNaN(value) to check for it rather than ===.

What is the difference between synchronous and asynchronous error handling?

Synchronous errors are caught with a regular try/catch around the code that might throw. Asynchronous errors (from Promises) are caught either with .catch() in a promise chain, or with a try/catch wrapped around an await expression inside an async function.

What is the difference between a for...in loop and a for...of loop?

for...in iterates over the enumerable property keys of an object (including arrays, where it gives indices). for...of iterates over the values of an iterable, such as an array, string, Map, or Set.

What is 'debouncing' and why is it useful?

Debouncing delays running a function until a certain amount of time has passed without it being called again, commonly used on search inputs so an API request only fires after the user stops typing rather than on every keystroke.

What is the difference between synchronous and asynchronous module loading (require vs. import)?

CommonJS's require() loads modules synchronously and is evaluated at the point it's called, which is why it can appear conditionally inside functions. ES Modules' import is statically analysed and hoisted to the top of the file, which allows tools to detect unused exports but means imports cannot be conditional in the same way.

What is a Set used for that a plain array isn't as well suited to?

A Set automatically enforces uniqueness and offers fast O(1) average lookups with has(), making it ideal for deduplicating values or checking membership in large collections without manually checking includes() on an array.

What does the spread operator do differently from the rest parameter, even though both use '...'?

Spread expands an existing iterable into individual elements (used when calling a function or building a new array/object), while rest gathers multiple individual arguments or remaining elements into a single array (used in a function's parameter list or in destructuring).

Why might you choose async/await over .then() chains?

async/await usually reads more like ordinary synchronous code, makes error handling simpler with a single try/catch block instead of a .catch() at the end of a chain, and is easier to combine with loops and conditional logic.

What is 'this' equal to in a regular function called without any object context (in non-strict mode)?

It defaults to the global object (window in browsers, global in Node.js). In strict mode, or inside ES modules and classes (which are strict by default), it is instead undefined.

Answer Key

Below is the correct answer and a short explanation for every multiple-choice question in this guide, grouped by part. Mark your own work honestly — if you got a question wrong, revisit that topic's theory section before moving on.

Part 0: Environment Setup

Q1: What does installing Node.js give you, beyond the ability to run JavaScript from the command line?
Correct answer: b) npm, for installing and managing packages — *Node.js ships bundled with npm (Node Package Manager), used to install and manage third-party JavaScript packages.*

Q2: Which command confirms that Node.js was installed correctly?
Correct answer: a) node -v — *node -v prints the installed Node.js version, confirming the installation succeeded and is on your PATH.*

Q3: What is the purpose of a .gitignore file containing 'node_modules'?
Correct answer: b) It prevents the large, regenerable node_modules folder from being committed to version control — *node_modules can be regenerated at any time with npm install, so committing it to Git is unnecessary and bloats the repository.*

Q4: Which VS Code extension automatically formats your code every time you save (once enabled)?
Correct answer: c) Prettier — *Prettier reformats your code consistently, and combined with the 'format on save' setting, does so automatically every time you save a file.*

Q5: What is the difference between a regular dependency and a devDependency in package.json?
Correct answer: b) devDependencies are only needed during development (e.g. testing tools), while dependencies are needed for the app to run — *npm install --save-dev (devDependencies) is for tools only needed while building/testing, such as nodemon or Jest, not for code your shipped app relies on at runtime.*

Part 1: Fundamentals

Q6: Which keyword should you use by default for a variable that will never be reassigned?
Correct answer: c) const — *const signals intent clearly and prevents accidental reassignment; use let only when a variable's value needs to change.*

Q7: What does typeof null return?
Correct answer: c) "object" — *typeof null returns "object" due to a long-standing historical bug in JavaScript that has been kept for backward compatibility.*

Q8: Which of these values is truthy?
Correct answer: c) [] — *Empty arrays are truthy. The only falsy values are false, 0, "", null, undefined, and NaN.*

Q9: What is the result of 5 === "5"?
Correct answer: b) false — *The strict equality operator (===) checks both type and value, and a number is never strictly equal to a string.*

Q10: Which loop is best suited to iterating over the values of an array?
Correct answer: b) for...of — *for...of iterates over the values of iterables such as arrays, strings, maps, and sets.*

Q11: What does the following return: Boolean(undefined ?? "default")?

Correct answer: a) true — *?? returns the right-hand value ("default") when the left side is null or undefined, and Boolean("default") is true.*

Q12: Which array method adds an element to the end of an array?

Correct answer: c) push() — *push() appends one or more elements to the end of an array and returns the new length.*

Q13: In an object literal, how do you access a property whose name is stored in a variable called key?

Correct answer: b) obj[key] — *Bracket notation evaluates the expression inside the brackets, so obj[key] looks up the property named by the variable's value.*

Q14: What is a function declaration's hoisting behaviour compared to a let variable's?

Correct answer: b) A function declaration is fully usable before its line runs; a let variable throws if accessed too early — *Function declarations are hoisted with their full definition, while let/const are hoisted into a temporal dead zone that throws on early access.*

Q15: Why does [1, 2] === [1, 2] evaluate to false?

Correct answer: c) === compares object references, and these are two distinct arrays in memory — *Objects and arrays are compared by reference; two separately created arrays are never === even with identical contents.*

Q16: What does "use strict" help catch?

Correct answer: c) Accidentally creating global variables and other silent mistakes — *Strict mode turns several previously-silent mistakes, like assigning to an undeclared variable, into thrown errors.*

Part 2: Intermediate

Q17: Which array method returns a brand-new array without modifying the original?

Correct answer: b) map() — *map() always returns a new array, leaving the original untouched, unlike sort() and splice(), which mutate in place.*

Q18: What value does reduce() return in nums.reduce((acc, n) => acc + n, 0) for nums = [1,2,3]?

Correct answer: b) 6 — *reduce() accumulates a single value by repeatedly applying the callback; starting from 0 and adding 1+2+3 gives 6.*

Q19: Why do arrow functions behave differently from regular functions regarding “this”?

Correct answer: b) Arrow functions do not have their own this and inherit it from the enclosing scope — *Arrow functions capture “this” lexically from where they are defined rather than how they are called.*

Q20: What does a closure allow a function to do?

Correct answer: b) Access variables from its outer scope even after that outer function has returned — *A closure keeps a reference to its surrounding scope's variables, which is how private state like counters is implemented.*

Q21: Which method converts a JavaScript object into a JSON string?

Correct answer: b) JSON.stringify() — *JSON.stringify() serialises an object into a JSON-formatted string; JSON.parse() does the reverse.*

Q22: In the DOM, which method selects ALL elements matching a CSS selector?

Correct answer: c) document.querySelectorAll() — *querySelectorAll() returns a NodeList of every matching element, while querySelector() only returns the first match.*

Q23: What is event delegation primarily used for?

Correct answer: b) Attaching one listener to a parent to handle events from many current or future children — *Delegation relies on event bubbling so a single parent listener can respond to events from child elements, including ones added later.*

Q24: Why does modifying shallow.address.city also change original.address.city after const shallow = { ...original }?

Correct answer: b) The spread only copies top-level properties; nested objects are still shared by reference — *The spread operator performs a shallow copy: nested objects/arrays inside the copy still point to the same memory as the original.*

Q25: What is the key difference between localStorage and sessionStorage?

Correct answer: b) localStorage persists until cleared; sessionStorage clears when the tab closes — *localStorage data survives across browser sessions until explicitly removed, while sessionStorage is scoped to a single tab's lifetime.*

Q26: What does Object.freeze() do to an object?

Correct answer: c) Prevents its properties from being changed, added, or removed — *Object.freeze() locks an object so its existing properties cannot be reassigned, and no new properties can be added.*

Part 3: Advanced

Q27: What does every JavaScript object have a hidden link to, forming the basis of inheritance?

Correct answer: b) A prototype — *Every object has an internal [[Prototype]] link to another object, and property lookups fall back to that prototype chain.*

Q28: What does the keyword 'super' do inside a subclass method?

Correct answer: b) Calls the corresponding method or constructor on the parent class — *super() calls the parent class's constructor, and super.methodName() calls the parent's version of a method.*

Q29: What does an async function always return?

Correct answer: c) A Promise — *Declaring a function async guarantees it returns a Promise, even if you write a plain 'return value;' inside it.*

Q30: Given the event loop rules, which callback runs first: a Promise.then() or a setTimeout(fn, 0)?

Correct answer: c) The Promise.then() callback (a microtask) runs first — *Microtasks (Promise callbacks) are always fully drained before the event loop processes the next macrotask, such as a timer.*

Q31: Which built-in structure guarantees unique values with no duplicates?

Correct answer: c) Set — *A Set automatically discards duplicate values when they are added, unlike an Array.*

Q32: In ES Modules, how do you import the default export from a file?

Correct answer: b) import anyName from './file.js'; — *A default export can be imported under any chosen name without curly braces, e.g. import multiply from './mathUtils.js';*

Q33: What problem do Promises and async/await primarily solve?

Correct answer: b) Deeply nested, hard-to-read callback chains ("callback hell") — *Promises and async/await let asynchronous operations be chained or written sequentially instead of nesting callbacks inside callbacks.*

Q34: What does currying transform a multi-argument function into?

Correct answer: b) A sequence of functions that each take one argument — *Currying restructures f(a, b, c) into f(a)(b)(c), returning a new function at each step until all arguments are supplied.*

Q35: Which of the following is a common cause of memory leaks in JavaScript?

Correct answer: b) Forgetting to clear an interval/timer or remove an event listener that is no longer needed — *Timers and listeners that are never cleaned up can keep closures (and everything they reference) alive indefinitely, preventing garbage collection.*

Q36: Which DevTools feature shows every network request your page makes, including status codes?

Correct answer: c) The Network tab — *The Network tab lists every HTTP request the page issues along with timing, status codes, and response bodies.*

Glossary of Key Terms

Term	Definition
API	Application Programming Interface — a defined way for one piece of software to talk to another, such as a web service.
Argument	The actual value passed into a function when it is called.
Asynchronous	Code that doesn't run immediately in sequence, but continues at a later point (e.g. after a network response).
Callback	A function passed into another function, to be executed later.
Closure	A function bundled together with references to its surrounding (outer) state.
DOM	Document Object Model — the browser's tree structure representing an HTML page.
ECMAScript	The official standard/specification that JavaScript implements.
Event loop	The mechanism that lets JavaScript run asynchronous callbacks after the current call stack is empty.
Falsy	A value that evaluates to false when used in a boolean context (false, 0, "", null, undefined, NaN).
Hoisting	JavaScript's behaviour of processing variable and function declarations before executing code line by line.
JSON	JavaScript Object Notation — a lightweight text format for storing and exchanging data.
Method	A function stored as a property of an object.
Module	A file of code that explicitly exports and imports functionality to/from other files.
npm	Node Package Manager — the default tool for installing and managing JavaScript packages.
Parameter	The named placeholder in a function definition that will receive an argument.
Promise	An object representing the eventual result (success or failure) of an asynchronous operation.
Prototype	An object that another object inherits properties and methods from.
Scope	The region of code where a given variable can be accessed.
Truthy	A value that evaluates to true when used in a boolean context (everything except the falsy values).
V8	The JavaScript engine created by Google, used in Chrome and Node.js.
Callback	A function passed into another function to be invoked later, often once an asynchronous task completes.
Currying	Transforming a function that takes multiple arguments into a chain of functions that each take one argument.
Debounce	A technique that delays running a function until a pause in repeated calls, commonly used for search inputs.

Term	Definition
Garbage collection	The JavaScript engine's automatic process of freeing memory used by objects that are no longer reachable.
Higher-order function	A function that takes another function as an argument, returns one, or both.
Immutability	The property of a value that cannot be changed after it is created; const and Object.freeze support this in JavaScript.
Pure function	A function that always returns the same output for the same input and produces no side effects.
Strict mode	An opt-in mode ('use strict') that makes JavaScript throw errors for certain unsafe or ambiguous actions.
Template literal	A string written with backticks that supports embedded expressions (${...}) and multi-line text.
Web API	Browser-provided functionality (like the DOM, fetch, or localStorage) that isn't part of the JavaScript language itself but is available to scripts running in a browser.